

Guion de sustentación: dominio y puertos/adaptadores

Parte del backend en la sustentación del taller (Grupo 2 · FAVM). Duración objetivo: **7 minutos + 3 de demo**. Diapositivas: [sustentacion-dominio-puertos.pptx](#) (también en PDF). Cada diapositiva trae este guion en las notas del orador.

Mensaje central (repetirlo al inicio y al cierre): las reglas de la adopción viven en el dominio y no dependen de ninguna tecnología; FastAPI y MongoDB son detalles intercambiables, y lo demostramos con pruebas que corren sin red.

Agenda y tiempos

#	Diapositiva	Tiempo	Idea que debe quedar
1	Portada	0:20	El núcleo no depende de la tecnología.
2	El dominio	1:00	4 entidades y 3 reglas; la regla 3 justifica Hexagonal.
3	Ciclo de vida	0:45	Las transiciones son métodos de la entidad; nadie se las salta.
4	Puertos	1:00	Protocolos definidos por el núcleo, pequeños y a la medida.
5	Adaptadores (C4)	1:15	Entrada, salida y un solo punto de cableado (<code>dependencies.py</code>).
6	Aprobar: de HTTP al dominio	1:00	Recorrido real de la regla 3, con el orden de escritura.
7	Evidencia: testabilidad	0:45	100 % del núcleo sin red, en $\approx$ 2 s.
8	Decisiones y costos	0:45	El costo del estilo es real y está justificado.
—	Demo en vivo	3:00	Ver sección siguiente.

Qué decir en cada diapositiva

- 1. Portada.** Vamos a mostrar el corazón del backend: el dominio, los puertos que definen qué necesita y los adaptadores que conectan FastAPI y la persistencia.
- 2. El dominio.** Cuatro entidades modeladas con Pydantic en `app/domain/entities.py`. Pydantic garantiza la forma (tipos, rangos, enums); las reglas son métodos. - *Regla 1:* el animal se valida a sí mismo (`validar_postulable()`). Decisión del equipo: solo `adoptado` bloquea; `postulado` sigue recibiendo postulaciones, porque si no nunca habría dos pendientes y la regla 3 no tendría sentido. - *Regla 2:* la valida el caso de uso `Postular` preguntando al puerto `existe_pendiente()`. Solo cuenta la pendiente: tras un rechazo se puede volver a postular. - *Regla 3:* `AprobarPostulacion` adopta el animal y cierra automáticamente las demás pendientes.

3. Ciclo de vida. Los estados son enums y cada transición es un método. `marcar_adoptado()` solo funciona desde `postulado`; resolver dos veces una postulación lanza `PostulacionYaResuelta`. El `motivoCierre` distingue si la rechazó el refugio o si se cerró sola.

4. Puertos. Un puerto es un contrato que define el núcleo con `typing.Protocol`: cualquier clase con esos métodos lo cumple sin heredar. `existe_pendiente()` existe solo para la regla 2: los puertos nacen de los casos de uso, no de la base de datos. Una prueba automática falla si el dominio importa infraestructura.

5. Adaptadores. Arriba, entrada: los routers traducen JSON:API a casos de uso y `errors.py` traduce excepciones a HTTP (409, 404, 422). En el centro, `dependencies.py` es el *composition root*: el único archivo que decide qué adaptador se inyecta con `Depends()`. Abajo, salida: in-memory (pruebas) y Mongo, con los mismos puertos. (En el diagrama los casos de uso llevan el sufijo `UseCase`; en el código se llaman `Postular`, `AprobarPostulacion`, `PublicarAnimal`.)

6. Aprobar. El router valida solo formato (ADR-006) → `Depends` entrega los repositorios → el caso de uso le pide a las entidades que cambien → se guarda primero el animal y la aprobación y al final el cierre automático (si algo falla a mitad, nunca quedan postulaciones cerradas sobre un animal sin adoptar) → la respuesta trae el animal y las cerradas en `included`.

7. Evidencia. Las pruebas del núcleo corren en un contenedor **sin red**: no hay Mongo ni servidor alcanzable, y aun así el 100 % del dominio y los casos de uso queda cubierto en ≈ 2 s. Con mocks verificamos interacciones: un 409 nunca llega a `guardar()`.

8. Decisiones y costos. Hexagonal tiene costo: más estructura que un CRUD para 4 entidades. Lo aceptamos porque el valor está en reglas de estado no triviales. Las imperfecciones que encontramos están en la periferia, ninguna en el dominio.

Demo en vivo (3 min)

Preparar antes: MongoDB arriba (`docker compose up -d mongodb`), la imagen de pruebas construida (ver [evidencia de testabilidad](#)) y dos terminales abiertas en `backend/`.

- 1. El núcleo sin infraestructura (40 s).** Mostrar que el dominio no importa FastAPI ni Motor, y correr las pruebas sin red: `bash docker run --rm --network none -v "$PWD:/app" adopcion-tests python -m pytest tests/unit -q --cov=app/domain --cov=app/application` Señalar: 100 % y ≈ 2 s.
- 2. Una regla fallando a propósito (60 s).** En `app/domain/entities.py`, cambiar `puede_recibir_postulaciones()` para que devuelva siempre `True` y volver a correr: fallan las pruebas de la regla 1 (unitarias y HTTP). Deshacer el cambio. Mensaje: la regla está en un solo lugar y está vigilada.
- 3. La API real (80 s).** Con el backend arriba, abrir `http://localhost:8000/docs` (Swagger generado por FastAPI) y mostrar el `PATCH /v1/postulaciones/{id}`; o correr la prueba de flujo completo: `bash docker run --rm -v "$PWD:/app" adopcion-tests python -m pytest tests/integration/test_api.py -k flujo_completo -v`

Plan B si Docker falla: mostrar las capturas de `docs/evidencias/testabilidad/` y el archivo `tests/unit/test_casos_de_uso_con_mock.py`.

Preguntas probables y respuestas

¿Por qué Pydantic en el dominio, si el ADR-006 dice que Pydantic valida en el borde? Son dos usos distintos. En el borde (`schemas.py`) Pydantic valida el *documento JSON:API*; en el dominio solo garantiza la *forma* de la entidad (tipos, rangos). Las reglas de negocio son métodos y lanzan excepciones propias. Si una entidad rechaza un dato, la API responde 422, no 500.

¿Qué diferencia hay entre un puerto y una clase abstracta? Un `Protocol` usa tipado estructural: el adaptador no hereda ni importa nada del núcleo, solo tiene los métodos. Eso mantiene la flecha de dependencia hacia adentro.

¿Por qué los puertos están en `application/ports` y no en `domain` ? Los puertos expresan lo que necesitan los *casos de uso* (capa de aplicación). El dominio (entidades y reglas) no necesita persistir nada. El ADR-007 menciona "en el dominio"; en el código quedaron en `application` , lo que respeta igual la regla de dependencias.

¿Qué pasa si dos adoptantes postulan al mismo tiempo, o dos personas aprueban a la vez? Con el adaptador in-memory no hay concurrencia real. En MongoDB, la regla 2 tiene respaldo en el índice único parcial `postulacion_pendiente_unica` ; el adaptador debe traducir `DuplicateKeyError` a `PostulacionDuplicada` . Para dos aprobaciones simultáneas haría falta actualización condicional (por estado o versión). Está documentado como pendiente en la revisión de validaciones.

¿Cómo se cambia de in-memory a Mongo? Escribiendo un adaptador que cumpla los cuatro puertos y cambiando `crear_repositorios()` en `dependencies.py` . Ni los routers ni los casos de uso cambian. Las mismas pruebas de casos de uso se pueden correr contra ambos adaptadores.

¿No es *overengineering* para 4 entidades? En parte sí, y lo decimos en la matriz de calidad. Se justifica por la regla de cierre automático y porque el valor del sistema está en la corrección de las reglas, no en el volumen de pantallas.

¿Por qué `postulado` sigue aceptando postulaciones? Si el animal pasara a no disponible con la primera postulación, nunca habría dos pendientes y la regla 3 nunca se ejecutaría. `postulado` significa "tiene al menos una pendiente".

¿Qué principios de diseño se ven en el código? Ver la [matriz de principios](#): la D de SOLID es la definición del estilo; hay contraejemplos honestos (un singleton y herencia en `_ConRepositorios`), todos en la periferia.

Cifras para citar

Dato	Valor
Entidades	4 (Refugio, Animal, Adoptante, Postulación)
Puertos	4 <code>Protocol</code>
Casos de uso	11
Pruebas totales	157 (unitarias + integración HTTP + integración con MongoDB)
Cobertura del núcleo	100 % (<code>domain</code> + <code>application</code>)

Dato	Valor
Tiempo de la suite unitaria	≈ 2 s, sin red